

BIG DATA ANALYTICS

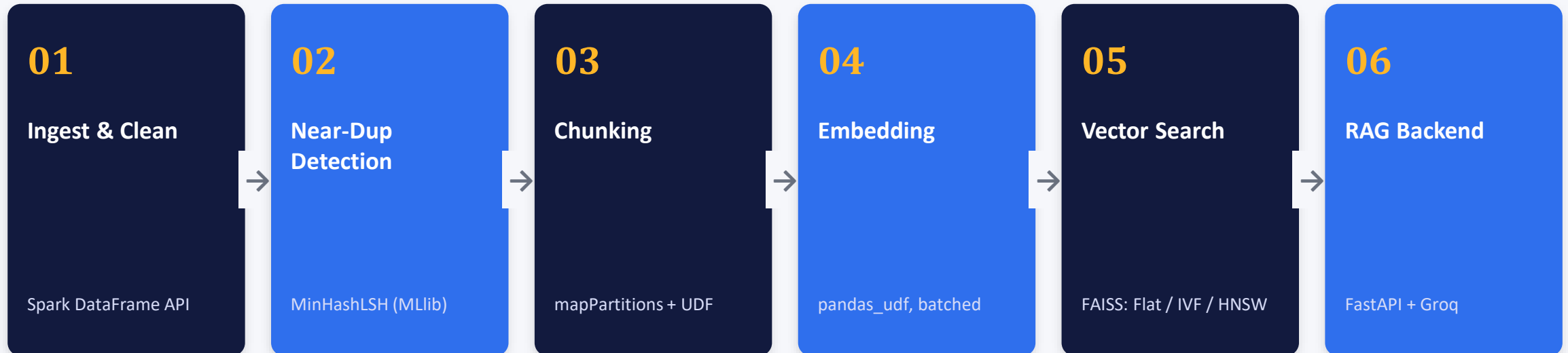


AI Assisted Laptop Finder

Spark Processing → FAISS ANN Retrieval → RAG Chatbot Backend

An end-to-end distributed data pipeline over 2,460 laptop listings, scraped from publicly available e-commerce sources — cleaning, near-duplicate detection, chunking and embedding at scale in Spark, ANN retrieval in FAISS, and grounded generation with Groq.

The Pipeline, End to End



Built and validated fully end-to-end, locally — Spark → Parquet → FAISS → FastAPI → Groq → Gradio — before any deployment step.

Merged Catalog: 2,460 Laptop Listings

Public dataset : [Laptops Price Dataset \(Gigasheet\)](#)

Scraped Dataset: Ryans computer ,Dazzle ,Global Brand

Final Preprocess Schema (10 columns)



2,460

Final Dataset

10

columns: structured specs + free text

0

exact duplicates on (name)

Cleaning & Near-Duplicate Detection

Cleaning — Spark DataFrame API

- Trim & collapse whitespace across all text columns
- Cast price to numeric; drop rows with no name / price
- Exact-dup removal on name

Near-Duplicate Detection — MinHash + LSH (MLlib)

- Shingle the compact spec_text
- (name+cpu+ram+storage+gpu+display) into 5-character shingles — not full marketing text, which is >90% boilerplate and floods results
- Threshold: Jaccard distance ≤ 0.15 ($\geq 85\%$ shingle overlap)
- Clusters resolved via union-find; keep lowest product_id per cluster

2,460

merged listings

2,460

after exact-dedup (0 removed)

584

near-duplicate pairs found ($\text{dist} \leq 0.15$)

2,057

final unique products (403 collapsed)

Text Chunking — mapPartitions + UDF

combined_unstructured_text runs to several hundred words per listing — too long to embed as one vector without losing detail. Each listing is split into overlapping word-window chunks.

120

words / chunk

20

word overlap

3,433

chunks produced

Two Spark implementations compared

- **mapPartitions (used downstream):**
operates on a whole partition as a Python iterator — no per-row UDF call overhead
- **UDF + explode() (shown for comparison):**
idiomatic DataFrame-land alternative, one UDF call per row — same result, different batching granularity

One listing → overlapping chunks

combined_unstructured_text (≈ several hundred words)

chunk_0 (120w)

chunk_1 (120w)

chunk_2 (120w)

*Shaded overlap (20 words) preserves context across chunk boundaries
— standard practice for RAG chunking.*

Distributed Embedding — pandas_udf

A `pandas_udf` receives Arrow-batched `pandas.Series` per call — enabling efficient batch-of-N forward passes, run in parallel across Spark partitions/executors. That parallel batching is the “distributed” part.

PREFERRED

all-MiniLM-L6-v2

- Sentence-Transformers neural model, 384-dim
- Loaded once per executor, reused across batches
- Hard load timeout + disk cache + `HF_HUB_OFFLINE=1` after first successful load, so no worker ever re-hits the network

OFFLINE FALLBACK

TF-IDF + Truncated SVD

- Used only if `huggingface.co` is unreachable
- Fit once on driver, broadcast to executors — a real LSA embedding, not a placeholder
- Same `pandas_udf` batching interface — swapping to a real transformer at deploy time is a one-line flag flip

384

embedding dimensions used

3,433

chunks embedded

278s

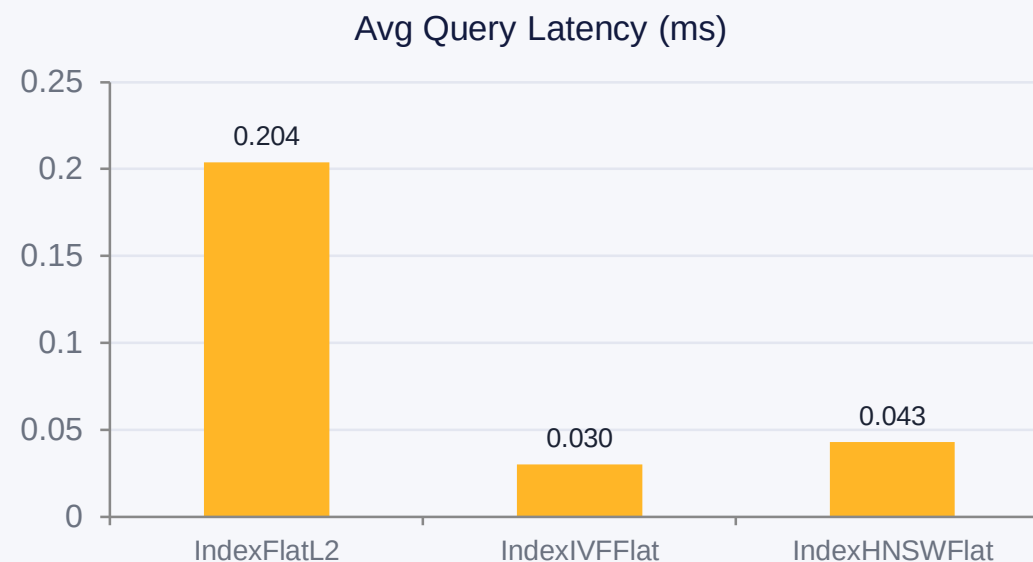
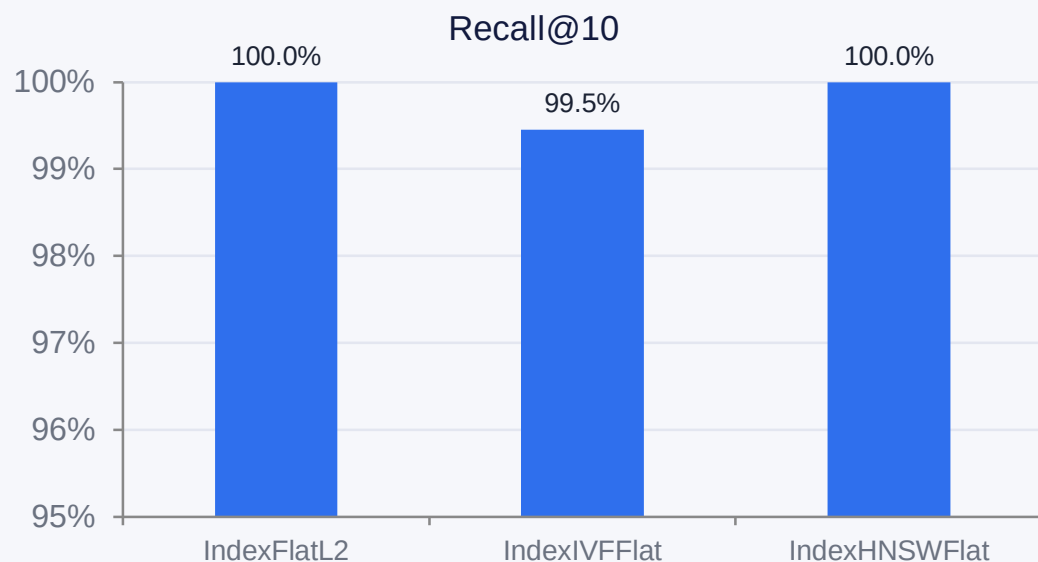
Spark embedding runtime

Parquet

embeddings + metadata output

Vector Search — Comparing 3 FAISS Indexes

200 sample queries over 3,433 vectors, $k = 10$. IndexFlatL2 (exhaustive) is ground truth for recall@10.

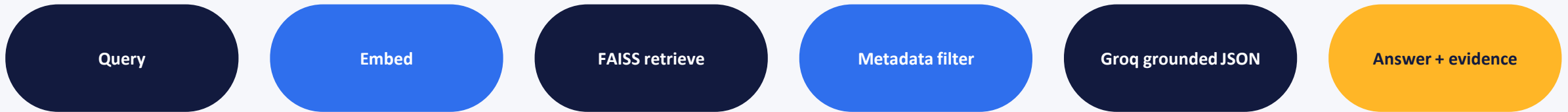


Decision: ship IndexFlatL2

At a few thousand vectors, exhaustive search is already sub-millisecond and always 100% recall — there's no accuracy/latency trade-off yet. IVF and HNSW are benchmarked and ready; the code re-points to either with a one-line swap once the catalog grows 50–100×.

RAG Chatbot Backend — FastAPI + Groq

Request flow



Grounding contract

- Groq may only recommend `product_ids` present in retrieved context — never invents a product, price, or spec
- Missing facts are written “Not specified”, not guessed
- Strict JSON schema: `answer`, `recommendations[]`, `unsupported_claim`
- Every hit carries a 0–100% `match_score` — exact cosine similarity derived from the L2 distance (both embedders L2-normalize)

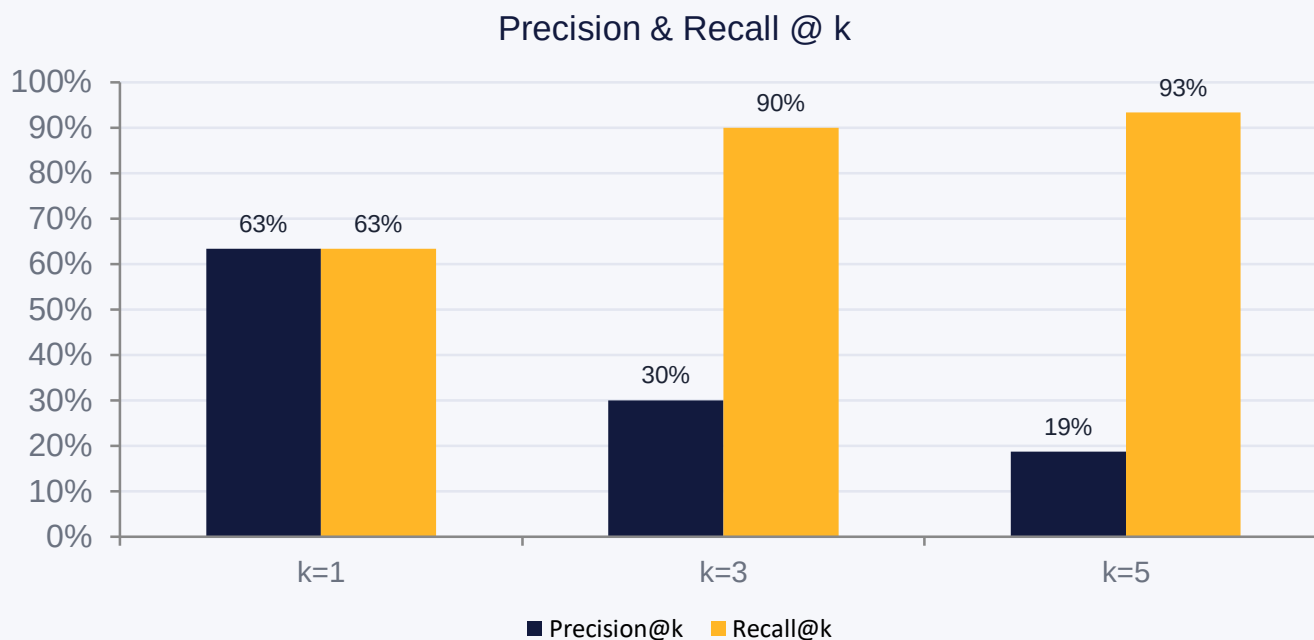
Reliability engineering

- Live model auto-selection (`client.models.list()`) instead of a hard-coded model name that Groq can deprecate
- Exponential backoff on rate-limit / connection errors (2s→4s→8s, capped 20s)
- Prompt trimmed to structured spec fields only — cut prompt tokens well below free-tier TPM limits
- API key read only from `GROQ_API_KEY` env var — never committed to notebook source

```
Live smoke test: mode = groq_rag · model auto-selected: openai/gpt-oss-120b · 3,433 vectors served over real HTTP (uvicorn, in-thread)
```


Retrieval Evaluation — 30 Hand-Labeled Queries

30 queries with expected_product_id labels drawn from this exact catalog. Precision/Recall computed at k = 1, 3, 5 via live FAISS retrieval (IndexFlatL2 shown — IVF and HNSW tracked identically at product level).

**94.9%**

avg match score @ k=1 (cosine sim.)

93.6%

avg match score @ k=5

< 1 ms

avg retrieval latency, all 3 indexes

Reading the curve: precision naturally falls as k grows (more slots, same one true label per query) while recall climbs toward 93% by k=5 — the correct product is almost always retrieved within the top 5.

Frontend — AI Laptop Advisor (Gradio)

Calls the FastAPI /chat endpoint and renders grounded product cards with an expandable evidence panel; a second tab surfaces the live evaluation charts and FAISS benchmark.

AI Laptop Advisor

Gaming laptop under 150000 BDT with dedicated GPU

Lenovo LOQ Gaming 15IAX9E — i5 12450HX

77% match

123,500 BDT

16GB RAM

512GB SSD

RTX 3050

15.6" FHD

Why it matches: price under 150k, dedicated RTX GPU, 16GB RAM, gaming-oriented model.

Retrieved Context / Evidence panel ▾

Two tabs

- **AI Laptop Advisor —**
natural-language search, price/brand/category filters, preset queries, product cards + evidence table
- **Evaluation & FAISS Results —**
dataset statistics, 30-query summary, FAISS benchmark table, precision/recall/latency charts

Live

public Gradio share link served straight from the notebook run

CONCLUSION

A Pipeline That Holds Up End to End

- 2,460 raw listings → 2,057 deduped products → 3,433 grounded, embedded chunks
- 3 FAISS indexes benchmarked; IndexFlatL2 chosen on evidence, not assumption
- Retrieval hits 93% recall@5 and ~94% average match score on hand-labeled queries
- Groq generation is strictly grounded to retrieved product_ids — no invented specs

Future Work

- Migrate to IndexIVFFlat / HNSW once the catalog reaches ~100K+ vectors
- Deploy the FastAPI service with uvicorn + Docker beyond in-process TestClient
- Swap in the neural sentence-transformer everywhere the sandbox forced the TF-IDF fallback
- Expand the hand-labeled evaluation set beyond 31 queries for tighter confidence intervals

Thank You

Questions & Discussion

CSE488 — Big Data Analytics | Laptop Product Intelligence RAG Pipeline

